



MESA CLICK

Análisis Integral de Base de Datos y Persistencia Transaccional

Motor de datos, transacciones ACID y arquitectura en tiempo real para gastronomía

Integrantes del Equipo

Martín Oviedo · Pablo Aguirre · Mateo Silvestrin

Carrera & Institución

Analista en Sistemas · I.R.E.S.M. (2026)

Stack Tecnológico

PostgreSQL 16+ · Go (pgx/v5) · Next.js · SSE





¿Qué datos maneja Mesa CLICK?

Antes de elegir un motor, analizamos a fondo la estructura y el flujo de los datos gastronómicos



Negocios y Sucursales

Multitenant estricto: marca, configuración fiscal, horarios de servicio y credenciales de cobro.



Sectores y Mesas QR

Salón, patio y terraza. Tokens QR únicos, versionado de cuenta y estado de ocupación en vivo.



Carta Gastronómica

Categorías, platos, precios y variantes con control rápido de stock momentáneo (código 86).



Comandas Colaborativas

Pedidos unificados por mesa y discriminados por comensal (recibido â preparando â listo).



Pagos y Transacciones

Cobro digital con Mercado Pago, división de consumos, cierres de cuenta y auditoría contable.



Usuarios y Roles

Admins, encargados y mozos con autenticación segura Magic Link sin contraseña vía JWT.

Conclusión del relevamiento: Los datos de Mesa CLICK están **fuertemente relacionados entre sí** y requieren **integridad referencial absoluta y transacciones ACID** (un pedido fallido o un plato agotado no debe registrar cobros ni comandas huérfanas en cocina).



Dos familias de bases de datos

Relacionales (SQL) frente a NoSQL orientadas a documentos

SQL Relacionales

POSTGRESQL · MYSQL

- **Tablas normalizadas:** Estructura fija y predecible con filas y columnas.
- **Integridad referencial:** Claves foráneas (FK) con borrado en cascada o restricción contable.
- **Transacciones ACID:** Garantía de 'todo o nada' en cobros y pedidos.
- **Concurrencia avanzada (MVCC):** Varios comensales en la misma mesa sin bloqueos de lectura.
- **Estándar SQL:** Consultas potentes con JOIN, agrupaciones y agregaciones analíticas.

NoSQL de Documentos

MONGODB · FIRESTORE

- **Documentos anidados (JSON/BSON):** Esquema flexible y dinámico.
- **Duplicación de datos:** Para cruzar datos obliga a desnormalizar o unir por código.
- **Integridad débil:** No hay claves foráneas en el motor; si falla la API quedan datos huérfanos.
- **Transacciones costosas:** Las transacciones multi-documento consumen excesiva memoria.
- **Escalabilidad horizontal:** Pensada para millones de lecturas distribuidas no relacionadas.



Cuadro comparativo: SQL vs. NoSQL

Evaluación rigurosa según los requerimientos operativos de Mesa CLICK

CRITERIO	SQL RELACIONAL (POSTGRESQL)	NOSQL DOCUMENTOS (MONGODB)
Modelo de datos	Tablas relacionales: calza exacto con Tenant + Sucursal + Mesa + Pedido.	Obliga a anidar pedidos en mesas o desnormalizar precios, arriesgando inconsistencias.
Integridad referencial	Claves foráneas y reglas CASCADE / RESTRICT protegen el historial de caja.	Sin claves foráneas nativas; la integridad queda librada al código de la API.
Transacciones (ACID)	ACID completo: registrar pedido, verificar stock (86) y comensal en 1 transacción.	Transacciones multi-documento lentas y con alto consumo en horas pico.
Concurrencia en Mesa	MVCC nativo: varios comensales agregan platos al mismo tiempo sin bloqueos.	Bloqueo de documento completo al actualizar pedidos en simultáneo.
Cierres de Caja & KPIs	Consultas SQL simples con SUM, AVG y GROUP BY en milisegundos.	Pipelines de agregación complejos; difícil cruzar facturación con artículos históricos.
Veredicto Final	Elección óptima para el Core transaccional	Descartada para la gestión operativa y financiera



Motores evaluados

Puntaje de 1 a 5 según los criterios esenciales del proyecto (5 = Excelente)

CRITERIO TÉCNICO EVALUADO	POSTGRESQL	MYSQL 8.4	MARIADB 11	MONGODB
Transacciones ACID & Aislamiento	5	4	4	2
Tipos nativos avanzados (UUID, JSONB, Time)	5	3	2	4
Integración y driver en Go (pgx/v5)	5	4	4	3
Aislamiento Multi-Tenant & Seguridad	5	4	4	2
Despliegue y compatibilidad Cloud (Render/Docker)	5	4	3	4
Licenciamiento 100% libre (Sin restricciones)	5	2	5	2
TOTAL (sobre 30 puntos)	30	21	22	17

Lectura del resultado: PostgreSQL alcanza la puntuación perfecta. Sus tipos nativos UUID (indispensables para los códigos QR), soporte JSONB para branding y el driver binario `pgx/v5` en Go lo posicionan como la solución técnica superior.



Comparativa técnica: PostgreSQL vs. MySQL vs. MariaDB

Análisis de capacidades de bajo nivel aplicadas a nuestra arquitectura

ASPECTO	POSTGRESQL 16+	MYSQL 8.4 LTS	MARIADB 11.8 LTS
Licencia	PostgreSQL License (100% abierta, sin dueño comercial)	Dual GPL v2 / Comercial (Propiedad de Oracle)	GPL v2 / LGPL (Fundación MariaDB)
Manejo de UUID	Tipo nativo `UUID` (128 bits binario eficiente + `gen_random_uuid()`)	BINARY(16) o VARCHAR(36) (conversiones manuales)	Soporta UUID, pero indexación menos optimizada
Datos Semiestructurados	`JSONB` binario descompuesto, indexable por GIN/B-Tree	JSON binario, sintaxis rígida en búsquedas	Alias de LONGTEXT (se guarda como texto plano)
Concurrencia & MVCC	Avanzado: lecturas no bloquean escrituras y control fino por fila	Bloqueos en metadatos y tabla de transacciones rígida	Similar a MySQL (motor InnoDB/Aria)
Driver en Golang	`jackc/pgx/v5`: protocolo binario directo y pool de alto rendimiento	go-sql-driver/mysql (Driver estándar database/sql)	Usa el driver de MySQL
Soporte en Render Cloud	Servicio nativo gestionado con backups automáticos	Requiere configuración externa o Dockerfile propio	No ofrecido nativamente en Render



Pros y contras para nuestro proyecto

Evaluación de ventajas y desventajas prácticas en la operación diaria

PostgreSQL (Elegido)

MOTOR DEFINITIVO

- âœ” **Transacciones ACID estrictas:** Evita comandas fantasmas o dobles cobros de mesas.
- âœ” **UUIDs Nativos:** Tokens QR seguros no predecibles (blindaje contra ataques IDOR).
- âœ” **JSONB de alto rendimiento:** Guarda estilos visuales y variantes sin crear 10 tablas extras.
- âœ” **Driver pgx/v5 en Go:** Latencia mínima (< 5ms) y manejo de pool de conexiones concurrente.
- âœ” **100% Open Source:** Sin riesgos de licencias comerciales ni compras corporativas.
- âœ– **Consumo de RAM base:** Ligeramente mayor que SQLite (despreciable en servidores cloud).

MySQL / MariaDB & NoSQL

ALTERNATIVAS

- âœ” **MySQL:** Muy popular y conocida en entornos educativos.
- âœ– **MySQL:** UUIDs requieren funciones o cadenas de 36 bytes ocupando más espacio e índice.
- âœ– **MySQL:** Propiedad de Oracle con funciones cerradas a la versión comercial Enterprise.
- âœ– **MariaDB:** JSON tratado como texto plano sin índices GIN; menor presencia en cloud gestionado.
- âœ– **MongoDB:** Falta de claves foráneas nativas; transacciones lentas y alto riesgo de inconsistencia de caja.



Decisión: PostgreSQL 16+ con Go (pgx/v5)

Los cuatro pilares arquitectónicos que fundamentan nuestra elección

1

Blindaje Transaccional

Cuando múltiples comensales envían pedidos o pagan por Mercado Pago simultáneamente, PostgreSQL resuelve la concurrencia garantizando integridad contable.

2

Seguridad con UUIDs

Todas las PKs son `'gen_random_uuid()'`. Impide que un cliente altere un ID numérico secuencial en la URL para visualizar o alterar pedidos de otras mesas.

3

Aislamiento Multi-Tenant

Particionado lógico por `'tenant_id'` con índices compuestos (`'idx_articulos_tenant'`). Cada bar o restaurante tiene sus datos completamente aislados de los demás.

4

Rendimiento con Go

El driver binario `'jackc/pgx/v5'` aprovecha al máximo el protocolo nativo de PostgreSQL, ofreciendo un pooling de conexiones robusto sin fugas de memoria.

Resultado en Producción: Servidor PostgreSQL corriendo en Render con 21 migraciones SQL automáticas ejecutadas al iniciar la API en Go, logrando cero latencia en salón.





Modelo Entidad-Relación de Mesa CLICK

Estructura normalizada en 13 tablas relacionales con aislamiento Multi-Tenant

Dominio 1 · Organización

6 tablas: tenants, usuarios, magic_tokens, sucursales, sectores y mesas.

Dominio 2 · Catálogo

3 tablas: categorias, articulos y variantes (con stock 86 y horarios).

Dominio 3 · Salón & Pagos

4 tablas: pedidos, pedido_items, pedido_item_variantes y pagos.

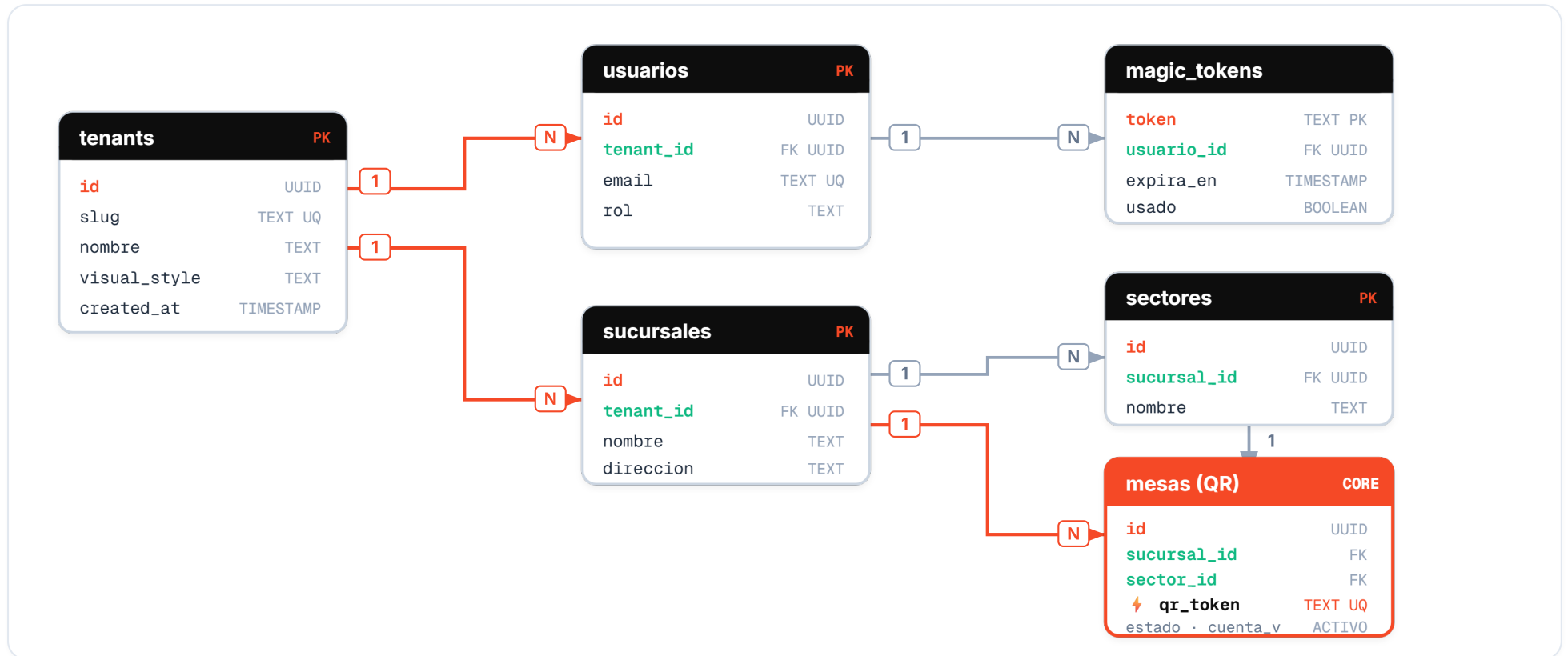
Estadísticas del esquema físico (PostgreSQL):

- **13 tablas relacionales** con 100% de identificadores basados en `UUID` nativo.
- **18 relaciones con claves foráneas (FK):** integridad asegurada en cascada (`ON DELETE CASCADE` para estructura y `RESTRICT` para auditoría contable).
- **16 índices especializados:** búsquedas optimizadas por token QR, combinaciones `(tenant\_id, estado)` y filtros de disponibilidad inmediata.



DER 1 • Multi-Tenant, Usuarios y Sucursales

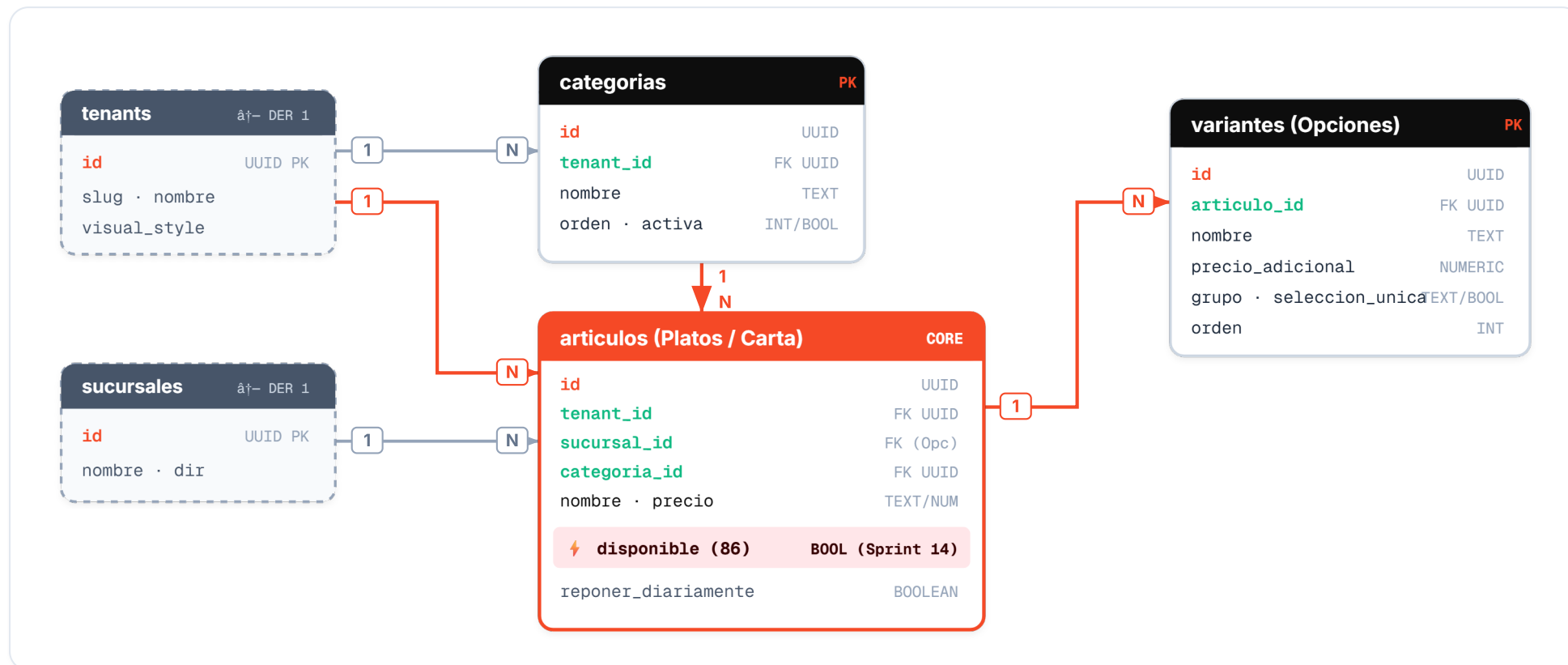
Estructura de cuentas de negocio, roles de acceso y distribución física de mesas con claves foráneas





DER 2 · Carta Gastronómica, Opciones y Disponibilidad (86)

Modelo del menú digital: categorías jerárquicas, platos con variantes y stock temporal

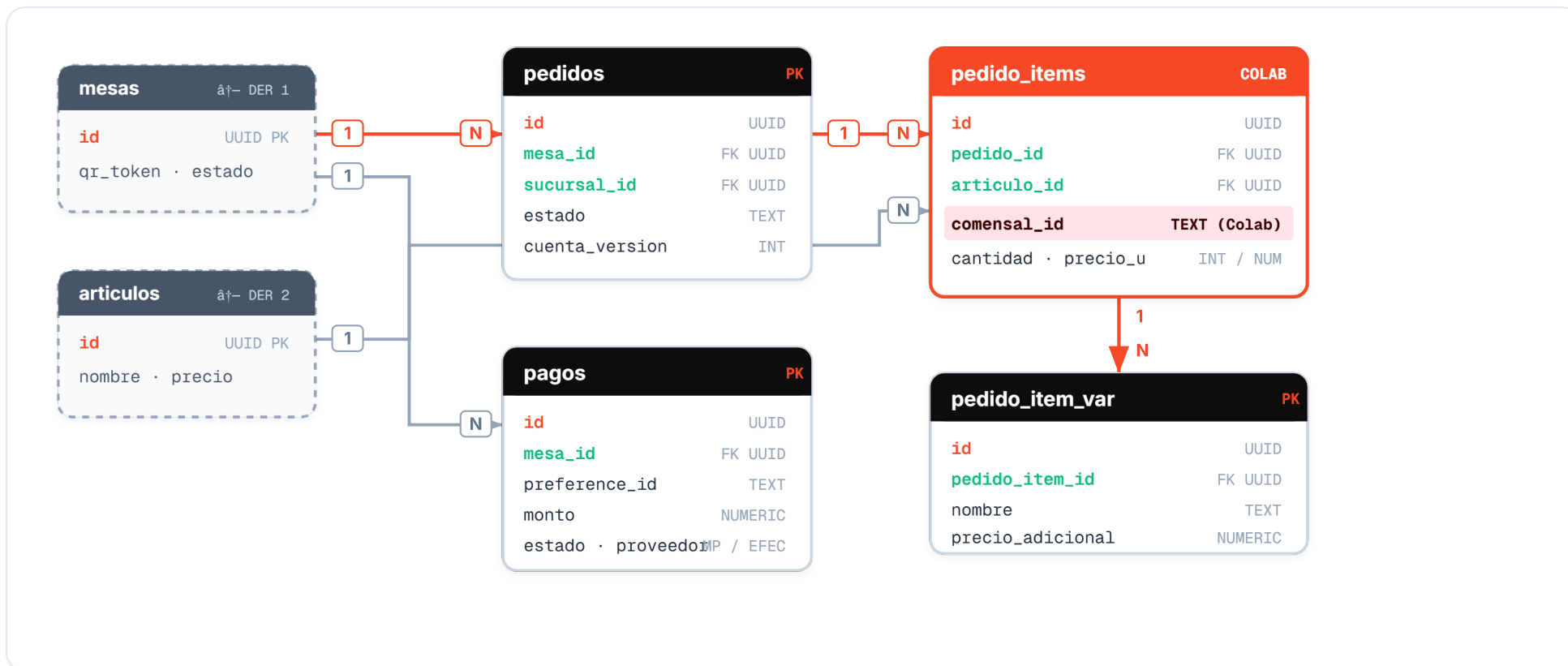


Innovación del Sprint 14: El atributo `disponible` permite marcar un plato como "86 Agotado" al instante en cocina. El comensal lo sigue viendo en la carta pero atenuado y con botón de compra deshabilitado, actualizándose por SSE sin recarga.



DER 3 · Operación de Salón, Comandas Colaborativas y Pagos

Ciclo transaccional de pedidos en mesa, desglose por comensal y cobros digitales



Soporte Colaborativo (US-81): Al asociar cada ítem a un `comensal\_id`, la base permite dividir la cuenta exactamente por persona o consolidarla en un único pago para toda la mesa sin duplicar registros.



Arquitectura de datos y eventos en tiempo real

PostgreSQL como fuente de verdad ACID y Broker SSE en Go para difusión instantánea

1. Comensal / Mozo

Envía pedido o cambia disponibilidad de stock desde el celular o dashboard.



2. Transacción ACID

PostgreSQL valida en bloque que los ítems estén disponibles y guarda el pedido.

3. Broker SSE (Go)

Al confirmar el commit, Go emite el evento en memoria a los canales suscritos.



4. Salón & Cocina

Pantalla KDS y mesas se actualizan en < 50ms sin hacer consultas continuas a la BD.

Eficiencia garantizada: Evitamos el problema común de saturación por *polling* (miles de peticiones HTTP por segundo). PostgreSQL solo trabaja cuando hay escrituras reales o consultas iniciales; la difusión masiva en tiempo real la resuelve Go en memoria.



Conclusión final

Resumen ejecutivo de la decisión tecnológica adoptada para Mesa CLICK



Base de Datos Relacional

Los datos gastronómicos son inherentemente relacionales. PostgreSQL 16+ garantiza integridad referencial indestructible, consistencia de caja y prevención de comandas huérfanas.

Seguridad & Multi-Tenancy

El uso extensivo de UUIDs nativos protege contra ataques IDOR en los códigos QR. El particionado por `'tenant_id'` permite que miles de locales convivan de forma segura en la misma base.

Rendimiento & Costo Cero

Conectado a Go mediante `'pgx/v5'` nativo y desplegado en Render con Docker, PostgreSQL ofrece máximo rendimiento bajo licencia 100% libre sin costos ocultos de Oracle ni MongoDB.

Proyecto Mesa CLICK (2026): Cumple al 100% con los estándares académicos y profesionales requeridos por el Instituto Remedios de Escalada de San Martín.